

COMUNIC@TE

ERP / POS para ventas, registros de equipos y autoservicio de clientes

Informe profesional de arquitectura, seguridad, rendimiento, Firebase, Netlify Functions, UX/UI, PDFs, calidad y despliegue.

PUNTUACION GENERAL	ESTADO DE LA AUDITORIA
68/100	DIAGNOSTICO COMPLETADO
Base funcional solida; riesgos de cuota, PII y escalabilidad pendientes.	Sin cambios de codigo durante esta auditoria.

Proyecto analizado

Version del proyecto: 6.2.5

Stack: React 19, Vite 7, Tailwind 3, Firebase Web/Admin, Firestore, Netlify Functions, jsPDF y Firebase Authentication.

Fecha del informe: 6 de agosto de 2026.

Nota de alcance

Este documento es un diagnostico tecnico basado en el codigo, configuracion, reglas, pruebas y dependencias disponibles en el repositorio. No se tuvo acceso directo a las metricas privadas de Firebase Console; por ello, las causas de cuota se presentan como hallazgos comprobables en codigo y deben contrastarse con Usage, Firestore Metrics y Netlify Analytics.

1. Resumen ejecutivo

COMUNIC@TE funciona como una aplicacion web interna para ventas, clientes, registros de equipos, consulta de IMEI, evidencias y autoservicio publico. La base actual es valida para una operacion pequena, pero la estrategia de almacenar imagenes en Firestore, la autorizacion compartida y el rate limiting por proceso representan limites importantes para crecer.

Dimension	Puntuacion	Lectura
Arquitectura	14/20	Buena separacion funcional, pero componentes y App.jsx concentran demasiadas responsabilidades.
Seguridad	11/20	Buenas validaciones server-side; faltan roles, verificacion Admin y proteccion distribuida.
Rendimiento	10/15	Code splitting y cache existentes; lecturas e imagenes en Firestore siguen siendo costosas.
UX/UI	12/15	Interfaz clara y operativa; pendientes de accesibilidad, feedback y densidad movil.
Calidad	9/15	52 tests backend pasan, pero faltan frontend, E2E, PDF visual y CI.
Escalabilidad	5/10	Adecuada para volumen pequeno; no preparada para multiples sucursales o gran PII.
Mantenibilidad	7/10	Estilo consistente, pero hay duplicacion, archivos grandes y documentacion desfasada.

Conclusiones prioritarias

- La principal causa tecnica probable de los excesos de cuota es guardar evidencias, fotos y logo como Base64 dentro de Firestore.
- El historial de DNI realiza una lectura completa para recalcular el contador de exitos en varias operaciones.
- Los cambios previos para reducir lecturas repetidas deben conservarse; el working tree mantiene unicamente esos tres archivos modificados.
- La aplicacion tiene validaciones server-side y transacciones importantes, lo cual es una base positiva para la siguiente etapa.

2. Hallazgos criticos

Codigo	Riesgo / impacto	Solucion propuesta
C1 - Imagenes en Firestore	Fotos de DNI, caja, recibos, evidencias y logo como Base64 causan escrituras grandes, lecturas pesadas y crecimiento rapido de cuota.	Firebase Storage privado; Firestore solo con metadata, hash, tamano y storagePath.
C2 - Conteo completo DNI Fotos	countSuccessEntries() recorre toda la coleccion en consulta, listado y eliminacion.	Contador agregado transaccional o conteo administrativo controlado.
C3 - Historial duplicado	DniFotosPage vuelve a consultar el historial aunque la respuesta ya trae los datos necesarios.	Actualizar estado local con la respuesta y refrescar solo cuando sea necesario.
C4 - Acceso compartido	Cualquier cuenta autorizada puede leer todos los datos personales del scope shared; no existen roles ni sucursales.	Custom Claims, roles, tenant/store isolation y reglas por accion.
C5 - Token validation	Backend usa Identity Toolkit y API key web en lugar de verifyIdToken con checkRevoked.	Firebase Admin verifyIdToken(token, true) y configuracion centralizada.
C6 - Rate limit local	El limite se reinicia por cold start y no protege contra trafico distribuido.	Rate limit persistente/distribuido y proteccion anti-bot.
C7 - Dependencias	npm audit reporta 1 critica, 4 altas y 8 moderadas en dependencias de produccion.	Actualizar por grupos, ejecutar tests/build/audit y fijar versiones seguras.

Evidencia principal: netlify/functions/dniFotos.mjs, netlify/functions/autoRegistros.mjs, src/app/App.jsx, netlify/functions/_shared.mjs, netlify/functions/_rateLimit.mjs y firestore.rules.

3. Hallazgos importantes

Area	Hallazgo	Prioridad
Firebase	Listeners de 200 clientes, 200 equipos, 40 registros y 40 ventas, mas conteos y busqueda historica limitada a 120 documentos.	Alta
Autoservicio	Las evidencias Base64 quedan asociadas a solicitudes; invitaciones eliminadas no tienen purga de evidencias.	Alta
Entradas	parseBody no aplica un limite general de body; Gemini valida presencia de imagen, pero no limite robusto de tamano/MIME.	Alta
Proveedores	RENIEC, Gemini y DNI Fotos usan fetch sin timeout/circuit breaker explicito.	Alta
Reglas	validRegistro y validVenta no reflejan todos los campos actuales del backend.	Media
Componentes	App.jsx y RegistroForm.jsx concentran autenticacion, IO, estado, validacion y render.	Media
Calidad	No hay pruebas React, E2E, visuales de PDF, accesibilidad automatizada ni CI visible.	Media

4. Hallazgos menores

- La lista de correos autorizados esta duplicada en frontend, backend y reglas Firestore.
- Hay patrones repetidos entre registros.mjs y ventas.mjs.
- La carga y compresion de imagenes se repite en los modulos de evidencias y PDF.
- Varios componentes escriben console.error directamente; conviene redirigir a un logger con redaccion de PII.
- ESLint termina sin errores, pero mantiene 8 warnings de hooks.
- Las notificaciones visuales no siempre incluyen aria-live o role=status.
- Algunos botones de icono requieren revisar el tamano tactil en movil.
- La pantalla de login mezcla textos en ingles y espanol.
- La documentacion enterprise contiene afirmaciones desfasadas respecto al codigo actual.

5. Plan de mejora priorizado

Prioridad	Acciones	Resultado esperado
Alta	Storage para evidencias, contador DNI Fotos, refresh sin duplicacion, Admin token verification, rate limit distribuido.	Menor cuota, menor superficie de ataque y mejor resiliencia.
Alta	Limites de body/MIME, timeouts externos, retencion y purga, actualizacion de vulnerabilidades.	Menos fallos operativos y menor riesgo de abuso.
Media	Roles, aislamiento por tienda, busqueda server-side, agregados y reglas alineadas.	Escalabilidad y control de datos.
Media	Refactor de App.jsx/RegistroForm.jsx, repositorios y pruebas E2E/PDF/a11y.	Mantenibilidad y calidad de entregas.
Baja	Documentacion, microinteracciones, textos, modal estandar y decision de tema oscuro.	Consistencia y pulido de producto.

6. Mejoras de rendimiento estimadas

Mejora	Estimacion	Comentario
Migrar evidencias a Storage	70-95 %	Reduce writes y tamaño de documentos Firestore.
Eliminar conteo completo DNI Fotos	90-99 %	Evita escanear toda la colección en cada respuesta.
Eliminar refresh duplicado	30-50 %	Reduce llamadas en el flujo de consulta.
Busquedas y agregados server-side	50-90 %	Evita escaneos por colección al crecer el historico.
Cache global de clientes	20-60 %	Disminuye consultas repetidas entre montajes.
PDF fuera del hilo principal	Mejor UX	Reduce bloqueos en telefonos de gama baja.

Las cifras son estimaciones de ingeniería, no mediciones de Firebase Console. Deben validarse antes y después con lecturas, escrituras, latencia p95, tamaño de payload y errores.

7. Mejoras de seguridad

Control	Prioridad	Recomendacion
Autenticacion	P0	Firebase Admin verifyIdToken(token, true), sin API key fallback en backend.
Autorizacion	P1	Custom Claims, roles y permisos separados para lectura, edicion, eliminacion y exportacion.
Privacidad	P1	Firebase Storage privado, URLs controladas, retencion y purga de documentos personales.
Abuso publico	P1	Rate limiting distribuido, CAPTCHA progresivo y limites por IP/token/DNI.
Entrada	P1	Limite de body, MIME allowlist, tamaño, dimensiones y validacion de Base64.
Proveedores	P1	Timeouts, circuit breaker, respuesta de error sanitizada y telemetria.
Headers	P2	Agregar CSP, HSTS y politica de cache para assets versionados.

Verificaciones positivas: React escapa texto por defecto; no se encontro dangerouslySetInnerHTML; los tokens aleatorios del portal se almacenan hasheados; existen validadores server-side y bloqueos de IMEI mediante transacciones.

8. Refactors recomendados

- Extraer useAuthSession, useFirestoreSubscriptions, useGlobalSearch, useBackupExport y useAppNavigation desde App.jsx.
- Separar RegistroForm.jsx en hook de formulario, validacion, cliente, equipo, precio y evidencias.
- Crear repositorios de dominio para clientes, equipos, ventas, registros y evidencias.
- Centralizar paginacion, contadores, auditoria, errores, rate limiting y subida de archivos.

9. Duplicacion de codigo

- Allowlist de usuarios repetida en src/config/auth.js, netlify/functions/_shared.mjs y firestore.rules.
- Patrones de transaccion, contador, delete/update y validacion repetidos entre registros.mjs y ventas.mjs.
- Carga y compresion de imagenes repetida entre registroEvidencias.js y registroEvidenciasPdf.js.
- Manejo de errores y consola distribuido en formularios y listas.

10. Archivos candidatos a refactor

Archivo	Motivo
src/app/App.jsx	966 lineas; estado, auth, listeners, busqueda, backup y navegacion.
src/features/registros/RegistroForm.jsx	1218 lineas; flujo complejo de cliente, equipo, precio y evidencias.
src/features/ventas/VentaForm.jsx	641 lineas; formulario, validacion, PDF y operaciones.
netlify/functions/dniFotos.mjs	Imagenes en Firestore, conteo completo y reconstruccion de chunks.
netlify/functions/registros.mjs	Patrones compartidos con ventas y logica de IMEI.
netlify/functions/ventas.mjs	Patrones compartidos con registros y logica de IMEI.

11. Dependencias a actualizar

Paquete	Estado observado	Accion
firebase	12.13.0; existe parche 12.x mas reciente.	Actualizar dentro de 12.x y probar cuota/auth.
firebase-admin	13.10.0; latest mayor 14.2.0.	Evaluar migracion mayor con pruebas de Netlify.
jspdf / dompurify	npm audit reporta vulnerabilidades transitivas.	Actualizar y verificar generacion de PDFs.
@grpc/grpc-js	Versiones transitivas con vulnerabilidades altas.	Actualizar mediante Firebase Admin/Firestore.
protobufjs / form-data	Vulnerabilidades altas transitivas.	Actualizar lockfile y repetir npm audit.
React / Vite	Parche disponible.	Actualizar en un cambio separado.

No se recomienda una actualizacion indiscriminada de Tailwind a la version 4 sin un plan de migracion visual y de build separado.

12. Riesgos futuros

- Exceso recurrente de cuota por Base64 en Firestore.
- Crecimiento rapido del almacenamiento de fotos y documentos personales.
- Exposicion de informacion entre sucursales por el modelo shared.
- Fallas de RENIEC, Gemini o proveedores externos sin timeout ni circuit breaker.
- Ataques automatizados al portal publico.
- Documentos Firestore demasiado grandes y PDFs pesados en moviles.
- Divergencia progresiva entre reglas Firestore y validadores backend.
- Dificultad de mantenimiento por archivos de mas de 1.000 lineas.
- Dependencias transitivas vulnerables no detectadas en cada deploy.

13. Roadmap tecnico

Horizonte	Entregables recomendados
0-2 semanas	Corregir countSuccessEntries; eliminar refresh duplicado; limites y timeouts; migrar evidencias criticas a Storage; corregir vulnerabilidades altas/criticas; pipeline basico.
2-6 semanas	Roles y Custom Claims; aislamiento por tienda; busqueda server-side; agregados; reglas alineadas; pruebas frontend/E2E/PDF/a11y; politica de retencion.
6-12 semanas	Observabilidad centralizada; PDFs fuera del hilo principal; backups verificados; panel de consumo Firebase; controles de privacidad; preparacion multi-sucursal.

Anexo A. Verificaciones ejecutadas

Comando / control	Resultado
npm test	52 pruebas correctas.
npm run lint	0 errores; 8 warnings preexistentes de hooks.
npm run build	Build de produccion correcto; bundles separados para Firebase, jsPDF, html2canvas y React.
git diff --check	Correcto.
npm audit --omit=dev	1 critica, 4 altas y 8 moderadas.
Cambios durante auditoria	Ninguno. Se conservaron los tres cambios previos de control de cuota.

Anexo B. Referencias de codigo

- src/app/App.jsx: listeners Firestore, conteos, busqueda historica, respaldo y logo.
- netlify/functions/dniFotos.mjs: imageChunks, conteo completo y respuestas de historial.
- netlify/functions/autoRegistros.mjs: evidencias Base64 y rate limit publico.
- netlify/functions/_shared.mjs: autenticacion por Identity Toolkit y pipeline comun.
- netlify/functions/_rateLimit.mjs: limite en memoria por instancia.
- firestore.rules: allowlist, modelo shared y contratos de campos.
- package.json: version 6.2.5, scripts de test/lint/build y dependencias.

Cierre

La recomendacion de mayor impacto es migrar todos los binarios fuera de Firestore y, en paralelo, corregir la autenticacion backend y el conteo global de DNI Fotos. Estas acciones atacan directamente los riesgos de cuota, seguridad y escalabilidad sin alterar el comportamiento funcional esperado del sistema.